

CoreData Consulting



Evidencia Elaboración de Pruebas de Software con uso de Jest.

Construcción de Software y Toma de Decisiones — Grupo 501
Instituto Tecnológico y de Estudios Superiores de Monterrey — Campus Querétaro

Integrantes del equipo

Luis Enrique de la Cruz Ambrosio — A01612863

Reflexión

Para el uso y creación de pruebas automatizadas, se hizo uso del framework de testing Jest, y uso de apoyo de IA para su implementación, esto debido a la poca experiencia en el manejo de este, para esto se hizo uso de Claude AI en su modelo Sonnet 4.6 para brindar un código guía base, el cual incluye comentarios y explicaciones sobre su implementación para los distintos Test Cases elaborados para las entregas de Avance. En base a estos la IA brindó una guía como la siguiente, la cual fue de gran utilidad para poder aprender más sobre su implementación y uso en distintos casos, y no quedarse únicamente con lo poco visto durante la única clase en la cual se abordó este tema.

JavaScript

```
// tests/operaciones.test.js
// Pruebas automatizadas para el módulo de Operaciones Financieras
// Casos cubiertos: CP-1 a CP-8
// Patrón: AAA (Arrange → Act → Assert)

// PASO 1: Importar la función utilitaria pura desde src/operacionUtils.
// Esta función NO usa Supabase, sólo recibe un objeto y retorna { valido,
// mensaje }.
const { validarCamposOperacion } = require('../src/operacionUtils');

// PASO 2: Mockear supabase ANTES de importar el modelo.
// Esto intercepta la conexión real a la BD y la reemplaza por funciones
// controladas desde cada test. El shape mínimo es { from: jest.fn() }.
jest.mock('../config/supabase', () => ({ from: jest.fn() }));

// PASO 3: Importar supabase y el modelo DESPUÉS del mock.
// Jest eleva jest.mock() al tope del archivo, así que el modelo ya
// recibirá la versión falsa de supabase sin importar el orden visual.
const supabase = require('../config/supabase');
const modeloOperaciones = require('../models/operaciones.modelo');

// =====
// BLOQUE 1 - validarCamposOperacion (función pura, sin BD)
// Cubre CP-1 a CP-6: todos los escenarios de campos faltantes o completos.
// =====
describe('operacionUtils - validarCamposOperacion', () => {

    // CP-1: Caso base negativo total.
    // Assert: valido === false Y mensaje contiene la palabra 'obligatorios'.
    test('CP-1 - todos los campos vacíos → inválido con mensaje de error', ()
=> {

        // Arrange - los 4 campos del formulario en blanco
        const datos = { idcliente: '', tipooperacion: '', monto: '', fecha: ''
};

        // Act
        const resultado = validarCamposOperacion(datos);
```

```

    // Assert
    expect(resultado.valido).toBe(/* TODO: ¿true o false? */);
    expect(resultado.mensaje).toContain(/* TODO: palabra clave esperada
*/);
  });

  // CP-2 a CP-5: Casos de campo faltante individual.
  // Patrón: dejar UN campo en '', los demás con valor válido.
  // Assert: sólo verificar valido === false (no hace falta revisar el
mensaje).
  test('CP-2 - falta idcliente, resto presentes → inválido', () => {
    // Arrange - completar los campos que SÍ tienen valor
    const datos = { idcliente: '', tipooperacion: '...', monto: '...',
fecha: '...' };

    const resultado = validarCamposOperacion(datos);

    expect(resultado.valido).toBe(/* TODO */);
  });

  test('CP-3 - falta tipooperacion, resto presentes → inválido', () => {
    const datos = { idcliente: '...', tipooperacion: '', monto: '...',
fecha: '...' };
    // TODO: Act + Assert siguiendo el mismo patrón de CP-2
  });

  test('CP-4 - falta monto, resto presentes → inválido', () => {
    const datos = { idcliente: '...', tipooperacion: '...', monto: '',
fecha: '...' };
    // TODO: Act + Assert siguiendo el mismo patrón de CP-2
  });

  test('CP-5 - falta fecha, resto presentes → inválido', () => {
    const datos = { idcliente: '...', tipooperacion: '...', monto: '...',
fecha: '' };
    // TODO: Act + Assert siguiendo el mismo patrón de CP-2
  });

  // CP-6: Caso positivo - todos los campos presentes.
  // Assert: valido === true Y mensaje === '' (cadena vacía, no undefined).
  test('CP-6 - todos los campos presentes → válido, mensaje vacío', () => {
    // Arrange - todos los campos con valores reales
    const datos = { idcliente: '...', tipooperacion: '...', monto: '...',
fecha: '...' };

    const resultado = validarCamposOperacion(datos);

```

```

        expect(resultado.valido).toBe(/* TODO */);
        expect(resultado.mensaje).toBe(/* TODO: cadena vacía */);
    });

});

// =====
// BLOQUE 2 – AgregarOperacion (modelo con Supabase mockeado)
// Cubre CP-7 y CP-8: respuesta exitosa vs. error de BD.
// =====
describe('AgregarOperacion - modelo (Supabase mockeado)', () => {

    // PASO 4: Fixture compartido con los 5 campos que el modelo espera.
    const datosValidos = {
        idcliente: '...',
        tipooperacion: '...',
        monto: '...',
        moneda: '...',
        fecha: '...'
    };

    // CP-7: Inserción exitosa.
    // Assert: resultado.exito === true Y resultado.operacion está definido.
    test('CP-7 - datos válidos y Supabase disponible → {exito: true} con operacion', async () => {
        // Arrange – simular la cadena from().insert().select() con éxito
        // La cadena de mocks refleja exactamente cómo el modelo llama a Supabase.
        supabase.from.mockReturnValue({
            insert: jest.fn().mockReturnValue({
                select: jest.fn().mockResolvedValue({
                    data: [{ /* TODO: objeto con los campos que devuelve la BD
*/ }],
                    error: null
                })
            })
        });

        // Act
        const resultado = await modeloOperaciones.AgregarOperacion(/* TODO */);

        // Assert
        expect(resultado.exito).toBe(/* TODO */);
        expect(resultado.operacion)./* TODO: matcher para verificar que existe
*/;

    });

```

```

// CP-8: Error de BD.
// Arrange: mismo shape de mock, pero data: null y error con mensaje.
// Assert: resultado.exito === false Y resultado.error está definido.
test('CP-8 - error de Supabase → {exito: false} con mensaje de error',
  async () => {
    // Arrange - misma cadena de mocks, ahora simulando fallo
    supabase.from.mockReturnValue({
      insert: jest.fn().mockReturnValue({
        select: jest.fn().mockResolvedValue({
          data: null,
          error: { message: '/* TODO: mensaje de error de BD */' }
        })
      })
    });

    const resultado = await modeloOperaciones.AgregarOperacion(/* TODO */);

    expect(resultado.exito).toBe(/* TODO */);
    expect(resultado.error)./* TODO: matcher para verificar que existe */;
  });
});

```

Referencias de uso de IA

- Anthropic. (2026). *Claude* (claude-sonnet-4-6) [Modelo de lenguaje de gran escala]. <https://claude.ai>
 Prompt: Toma el rol de un experto en Desarrollo de Software y Testing, Como implementar y cuales son los mejores casos de prueba a realizar para el testing de un login, y el registro de un cliente dentro de una aplicacion web, en el caso del registro estamos haciendo uso de un wizard, para esto se dividió en 3 pasos, lo cuales incluyen el registro de datos generales, carga de documentos y finalmente el resumen y guardado de datos, para esto explicame la mejor forma de dividir y mantener un orden en sus casos de prueba, también para una mejor explicación dame algunos ejemplos lo cuales incluyan ID, Descripción, Precondición, Entrada, Accion y Salida esperada, Link Chat: <https://claude.ai/share/d01c5990-9b09-43d2-b088-416769b6e343>
 Nota: En este también se incluye investigación extra para uso de JEST en el proyecto y casos de prueba.

Prompt IA para ayuda en ideación y creación de casos de prueba para los distintos apartados.

En base a los siguientes códigos ayúdame a idear casos de prueba para cada uno de estos apartados que son Alertas PLD, Reglas PLD y Operaciones, esto

principalmente ayudame a identificar las salidas válidas esperadas de cada uno de estas pruebas principalmente las cuales sean más complejas, así como también explica la importancias de estos, en el caso de estos los cuales llevan múltiples pasos como el caso de alertas el cual también incluye la necesidad de un cálculo de nivel de alerta y su resolución, dame cuales serian cada uno de estos puntos extra para cada parte como lo mencione antes, también ten en cuenta que actualmente estoy manejandolo en formato de tabla las cuales incluyen apartados importantes como lo son Descripción, Precondición, Entrada, Acción y Salida esperada , para eso explicame también el por que de cada apartado en los casos de prueba que recomiendes e ideas, para poder realizar estos de la mejor forma y haciendo uso de mejores prácticas al realizar estos

Prompt de IA para ayuda con la implementacion de Test con uso de Jest

Tengo un proyecto Node.js con arquitectura MVC que usa Express, EJS y Supabase como base de datos. Necesito que me ayudes a dando una guia de como implementar pruebas automatizadas con Jest para siguientes módulos: Login, Registro de Cliente, Operaciones Financieras, Alertas PLD y Reglas de Alertas. Esto basado en el documento de TestCases dado.

CONTEXTO DEL PROYECTO

Estructura de carpetas:

- controllers/ — login.controller.js, clientes.controller.js, operaciones.controller.js, alertas.controller.js, reglas.controller.js
- models/ — login.model.js, clientes.model.js, operaciones.model.js, alertas.model.js, reglas.model.js
- src/ — aquí puedes crear archivos de utilidades puras si los necesitas
- tests/ — aquí van los archivos de prueba
- config/supabase.js — cliente de Supabase (este debe ser mockeado en todos los tests)

REQUERIMIENTOS DE LAS PRUEBAS

1. Jest el cual ya está instalado.
2. Patrón Arrange — Act — Assert con comentarios explícitos en cada sección.
3. Mockear Supabase con jest.mock para evitar conexiones reales a la BD.
4. Numerar cada test con el ID del caso de prueba según el documento (CP-1, CP-2... o TC-RC-001, TC-RC-002...).

MÓDULOS Y CASOS A CUBRIR

Estos se encuentran dentro del documento de TestCases

NOTAS ADICIONALES

Para esto solo dame una base y guía de implementación con comentarios sobre como debe ir estructurado y partes las cuales sean muy importantes y dificiles de idear para su implementacion.

